

SIMATS ENGINEERING

ASSESSMENT TOOL 2 : Scenario-Based

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15 16

COURSE OUTCOME COVERED

CO2: *Analyze virtualization architectures to identify performance and security issues in cloud computing environments. (BL3)*

Dave's Taxonomy: Manipulation (Level 2)
Question Count: 5

Total Marks: 25

Code of Conduct

I S Tejayadav(192525447) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: S Teja yadav

Assessment Tasks

Scenario-Based Questions

1. A media company wants to migrate its video streaming platform to the cloud. The system must handle millions of concurrent users with minimal buffering. Design a cloud architecture using scalability and caching techniques. Explain how load balancing and auto-scaling can ensure smooth streaming. Discuss cost optimization strategies for such high-demand workloads.
2. A fintech startup is deploying applications using containers across multiple cloud providers. They face challenges in maintaining consistency and managing deployments. Propose a solution using container orchestration and multi-cloud strategies. Explain how service discovery and scaling can be handled efficiently. Highlight the benefits and risks of a multi-cloud deployment model.
3. An e-learning platform experiences data loss due to accidental deletion in the cloud. The organization needs a robust backup and disaster recovery strategy. Design a solution using cloud-native backup, replication, and versioning.

Explain how recovery time objective (RTO) and recovery point objective (RPO) are achieved.
Provide steps to ensure business continuity during failures.

4. A government agency is adopting a hybrid cloud model for sensitive and non-sensitive data.
Some applications must remain on-premises due to compliance requirements.
Design a hybrid architecture that ensures secure communication between environments.
Explain how identity management and encryption should be implemented.
Discuss challenges in managing hybrid cloud environments.
5. A SaaS provider needs to ensure high availability and fault tolerance for its application.
The system must continue operating even if one region goes down.
Propose a multi-region deployment strategy in the cloud.
Explain how data replication and failover mechanisms should be configured.
Discuss monitoring and alerting strategies for proactive issue detection.

Scenario-Based Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Scenario	20%	Demonstrates complete understanding of the scenario context	Good understanding with minor gaps	Partial understanding	Fails to understand the scenario
Identification of Risks / Issues	20%	Identifies all major issues correctly	Identifies most issues	Limited issue identification	Misses major issues
Application of Concepts	25%	Applies virtualization concepts effectively	Applies concepts with minor mistakes	Partially correct application	Weak application
Decision Justification	20%	Decisions are well justified and technically strong	Reasonable justification	Weak justification	No useful justification
Clarity	15%	Clear and well-organized response	Generally clear	Somewhat unclear	Poorly presented

1. Cloud Architecture for a Large-Scale Video Streaming Platform

A media company that provides video streaming services to millions of concurrent users requires a highly scalable, reliable, and low-latency cloud architecture. The major objectives are to reduce buffering, handle sudden increases in traffic, and provide continuous availability. A suitable architecture consists of **cloud object storage, Content Delivery Network (CDN), load balancers, auto-scaling servers, caching, and monitoring services.**

The basic architecture can be represented as:

Users → DNS/Global Traffic Manager → CDN → Load Balancer → Auto-Scaling Application Servers → Database/Object Storage

Videos should be stored in highly durable **cloud object storage** because video files are large and require reliable storage. Instead of sending every video request directly to the origin server, a **CDN** should be used. A CDN stores frequently accessed videos at geographically distributed edge locations. When a user requests a popular video, the CDN can deliver it from a nearby edge server. This reduces latency, bandwidth consumption, and load on the main cloud infrastructure.

Caching is another important technique. Frequently accessed video content should be cached at CDN edge locations. Application data such as video titles, categories, recommendations, and user preferences can also be cached using an in-memory caching system. A high CDN cache-hit ratio reduces the number of requests reaching the origin servers.

A **load balancer** distributes incoming requests among multiple application servers. Instead of allowing one server to handle millions of requests, traffic is distributed across many servers. Health checks can identify failed servers and automatically stop sending traffic to them.

Auto-scaling allows the infrastructure to dynamically increase or decrease resources according to demand. For example, during a popular sports event, millions of users may access the platform simultaneously. Auto-scaling can automatically add more application instances. When demand decreases, unnecessary instances can be removed. This provides both performance and efficient resource utilization.

The platform should also use **adaptive bitrate streaming**. Multiple versions of the same video can be stored at different resolutions and bitrates, such as 1080p, 720p, 480p, and 360p. The streaming application automatically selects an appropriate quality according to the user's network speed. This helps minimize buffering when network conditions change.

Cost optimization is important because continuously operating large numbers of servers can be expensive. The company can use auto-scaling to avoid idle resources, CDN caching to reduce origin bandwidth, storage lifecycle policies to move old videos to cheaper storage, and reserved or committed resources for predictable workloads. Non-critical video processing can use lower-cost compute resources where appropriate.

The company should continuously monitor CPU utilization, memory, network bandwidth, request latency, buffering rate, CDN cache-hit ratio, error rate, and concurrent users. Alerts should be generated when performance falls below acceptable levels.

Conclusion: A combination of **CDN, caching, load balancing, auto-scaling, object storage, adaptive bitrate streaming, and monitoring** can support millions of concurrent users while minimizing buffering. Proper cost management ensures that the system remains economically efficient even under very high demand.

2. Multi-Cloud Container Deployment for a Fintech Startup

A fintech startup deploying applications across multiple cloud providers may face problems such as inconsistent environments, deployment difficulties, different configurations, networking challenges, and complicated management. A suitable solution is to use **containers with a container orchestration platform such as Kubernetes**, together with standardized CI/CD and multi-cloud management practices.

The proposed architecture can be represented as:

Developer → CI/CD Pipeline → Container Registry → Kubernetes Cluster

Cloud Provider A | Cloud Provider B | Cloud Provider C

Containers package the application together with its libraries and dependencies. Therefore, the same container image can run consistently across different cloud environments. This reduces the common problem of an application working correctly in one environment but failing in another.

A **container orchestration platform** manages the lifecycle of containers. Kubernetes can automatically deploy containers, restart failed containers, distribute workloads, perform health checks, expose services, and scale applications. Instead of manually managing hundreds of containers, administrators define the desired state and Kubernetes works to maintain that state.

To maintain consistency, the company should use a **CI/CD pipeline**. Whenever developers update an application, the pipeline can build a new container image, perform automated tests, scan the image for vulnerabilities, store it in a container registry, and deploy the approved version to the required cloud environments. Infrastructure and configurations should also be maintained as code so that the same deployment process can be reproduced across clouds.

Service discovery is necessary because microservices frequently communicate with each other. Kubernetes provides services and DNS-based service discovery. Applications can communicate using stable service names rather than depending on changing container IP addresses. For communication between services across different cloud environments, API gateways, secure networking, or service-mesh technologies can be used.

Scaling can be handled using Kubernetes autoscaling. Horizontal scaling increases the number of application instances when CPU, memory, request rate, or other metrics increase. When demand decreases, unnecessary instances can be removed. Cluster-level scaling can also provide additional infrastructure capacity when required.

Multi-cloud deployment has several advantages. It can reduce **vendor lock-in**, improve availability, provide geographic flexibility, and allow the company to use suitable services from different cloud providers. If one provider experiences a major outage, another cloud environment may be used for continuity.

However, multi-cloud also creates risks. Managing several providers increases operational complexity. Networking between clouds can be difficult, and data transfer may create additional costs. Identity and access management must be integrated across environments. Keeping databases synchronized can also be challenging. Security policies, monitoring, logging, and compliance controls must be consistently applied to every cloud.

Conclusion: A standardized **container and Kubernetes-based multi-cloud architecture**, supported by CI/CD, service discovery, autoscaling, centralized monitoring, and common security policies, provides portability and scalability. Although multi-cloud increases management complexity, it can provide better resilience and reduce dependence on a single cloud provider.

3. Backup and Disaster Recovery Strategy for an E-Learning Platform

An e-learning platform stores important information such as student records, examination results, assignments, course materials, user accounts, and videos. Accidental deletion or infrastructure failure can cause serious data loss. Therefore, the organization should implement a strong **cloud-native backup and disaster recovery strategy** using automated backups, versioning, snapshots, replication, and a separate recovery environment.

The basic architecture can be represented as:

Primary Application → Primary Database/Object Storage → Automated Backup → Versioned Backup → Secondary Region

Database backups should be performed automatically. The organization can use full backups periodically and incremental backups more frequently. Database transaction logs can also be backed up at short intervals. This allows the organization to restore data to a recent point in time.

Storage versioning is particularly useful for accidental deletion. If a teacher or administrator accidentally deletes a course document or video, an earlier version can remain available. The organization can restore the previous version instead of permanently losing the data. Versioning also protects against accidental overwriting of important files.

Critical data should be **replicated to another availability zone or geographic region**. For example, if the primary cloud region becomes unavailable because of a major infrastructure failure, the organization can use the replicated data in a secondary region. Cross-region replication therefore improves disaster recovery and business continuity.

Two important disaster recovery concepts are **RTO and RPO**.

Recovery Time Objective (RTO) is the maximum acceptable time required to restore a service after a failure. For example, if the RTO is 30 minutes, the e-learning platform should be restored within 30 minutes. RTO can be achieved using standby infrastructure, automated deployment, preconfigured recovery environments, health checks, and automated failover.

Recovery Point Objective (RPO) specifies the maximum acceptable amount of data loss, measured in time. For example, an RPO of 5 minutes means that the organization should not lose more than approximately five minutes of data. Frequent backups, database replication, and transaction-log backups help achieve a low RPO.

Business continuity should follow several steps. First, the organization should identify critical applications and data. Second, suitable RTO and RPO values should be defined. Third, automated backups and storage versioning should be enabled. Fourth, important data should be replicated to another region. Fifth, a standby recovery environment should be maintained. Finally, automated failover and regular disaster recovery testing should be implemented.

Backups should be encrypted and protected using strong access controls. Backup administrators should have only the permissions necessary to perform their duties.

Conclusion: A combination of **automated backups, storage versioning, snapshots, cross-region replication, standby infrastructure, encryption, and regular recovery testing** provides strong protection against accidental deletion and infrastructure failures. RPO is achieved through frequent backup and replication, while RTO is achieved through rapid restoration and automated failover.

4. Secure Hybrid Cloud Architecture for a Government Agency

A government agency may need to adopt a **hybrid cloud architecture** because some information must remain within the organization's on-premises infrastructure due to compliance, security, or legal requirements. At the same time, non-sensitive applications can benefit from the scalability and flexibility of public cloud services.

The proposed architecture is:

Users → Identity Management → Secure Gateway → On-Premises Datacenter ↔ Secure Connection ↔ Public Cloud

Sensitive applications and databases should remain in the **on-premises environment**. These may include confidential government records, sensitive citizen information, regulated databases, and applications that are subject to strict compliance requirements.

Non-sensitive workloads can be hosted in the **public cloud**. Examples include public websites, general information systems, collaboration applications, and suitable analytics workloads. This division allows the agency to obtain cloud benefits without unnecessarily moving sensitive information outside its controlled environment.

Secure communication between on-premises and public cloud systems is essential. The agency can use a **site-to-site VPN or dedicated private connection**. Data travelling between environments should be encrypted. Firewalls, network segmentation, access-control rules, and security gateways should restrict communication so that only authorized applications and users can access required resources.

Identity management should be centrally controlled. The organization should use strong authentication, **multi-factor authentication (MFA)**, role-based access control, and the principle of least privilege. Identity federation can allow users to access authorized cloud and on-premises resources using trusted identities rather than maintaining separate accounts everywhere. Privileged administrator accounts should receive additional protection and monitoring.

Encryption should be applied both to data in transit and data at rest. Communication between the two environments should use encrypted connections. Databases, files, and backups containing sensitive information should also be encrypted. Encryption keys must be securely managed, and access to keys should be restricted.

Network security should include firewalls, intrusion detection and prevention systems, security groups, private endpoints, and network segmentation. Sensitive systems should not be directly exposed to the public internet.

The main challenges of hybrid cloud include **management complexity, network connectivity, identity integration, compliance requirements, security monitoring, cost management, and the need for skilled personnel**. The agency must maintain consistent security policies across both environments. Monitoring and centralized logging should be used to detect unauthorized access, unusual network activity, configuration changes, and security incidents.

Conclusion: A secure hybrid cloud architecture allows government agencies to keep sensitive applications on-premises while using public cloud resources for non-sensitive workloads. Secure connectivity, IAM, MFA, encryption, network segmentation, monitoring, and centralized security policies are essential. Although hybrid cloud increases management complexity, it provides a good balance between **security, compliance, scalability, and flexibility**.

5. Multi-Region Deployment for High Availability and Fault Tolerance

A SaaS provider serving many customers must ensure that its application remains available even if an entire cloud region becomes unavailable. A **multi-region deployment** provides high availability by running application infrastructure in two or more geographically separated cloud regions.

A suitable architecture is:

Users → Global Traffic Manager → Region A / Region B → Application Servers → Replicated Database

Each region should contain multiple application instances, load balancers, storage, and other required services. Running multiple application instances ensures that the failure of one server does not make the application unavailable.

A **global traffic manager** or DNS-based traffic-management system can direct users to a healthy region. Traffic may be distributed based on location, latency, capacity, or health status. Continuous health checks should monitor each region. If Region A fails, the traffic manager should automatically redirect users to Region B.

Application servers should preferably be **stateless**. Session information should be stored in a shared or replicated service rather than only on one server. This makes it easier to move users between regions during failover.

Database replication is a critical part of multi-region deployment. Data from the primary region should be replicated to the secondary region. Synchronous replication provides stronger consistency but can increase latency, while asynchronous replication generally provides better performance but may result in a small amount of data loss during sudden failures.

The SaaS provider can use either an **active-passive** or **active-active** model. In active-passive architecture, one region normally serves users while the other remains ready for disaster recovery. In active-active architecture, both regions serve users simultaneously. Active-active can provide better availability and resource utilization, but it is more complex because data consistency and conflict management become important.

Monitoring should continuously track **CPU, memory, application latency, request errors, database health, storage capacity, replication lag, and availability**. Alerts should be configured for critical conditions such as regional failure, high error rates, increased latency, and database replication problems.

The organization should regularly conduct **disaster recovery and failover tests**. A controlled regional failure can be simulated to verify that traffic moves correctly to the secondary region and that the application and database continue functioning.

A multi-region architecture provides high availability, fault tolerance, disaster recovery, and potentially lower latency for geographically distributed customers. However, it also increases infrastructure cost, cross-region data-transfer costs, deployment complexity, and database-management requirements.

Conclusion: A SaaS provider can achieve high availability using **multi-region application deployment, global traffic management, redundant servers, database replication, automated health checks, failover, monitoring, and alerting**. Although multi-region deployment is more complex and expensive, it provides strong protection against regional failures and is suitable for mission-critical SaaS applications.